



Informatics II

Exercise 3

Grgic Anto

Schedule (Corrected)

Date	Topic
27. Feb	C environment
06. Mar	Ex1(Introduction to C, basic sorting)
13. Mar	Ex2 (Recursion)
20. Mar	Ex3 (Complexity, Correctness)
27. Mar	Ex4 (Divide and Conquer, Recurrence)
03. Apr	NO LAB (Please attend another class) – Spring break
17. Apr	Ex6 (Pointers, Linked Lists)
24. Apr	Ex7(ADTs, Stacks, Queues)
01. May	NO LAB (Please attend another class) – Labor day
08. May	Ex9 (Hash Tables)
15. May	Ex10 (Dynamic Programming)
22. May	Ex11 (Graphs)
29. May	Recap
04. Jun	Exam at Messe 16:00-19:00

Binary search

- Array needs to be sorted
- Time complexity of $O(\log(n))$
 - Is that good? Why?
- The idea is to delete half of the possibilities on each iteration of the algorithm by looking at the middle of the array

n	$\log_2(n)$
1	0
10	3
100	7
1,000	10
10,000	13
100,000	17
1,000,000	20
10,000,000	23
100,000,000	27
1,000,000,000	30
10,000,000,000	33
100,000,000,000	37
1,000,000,000,000	40

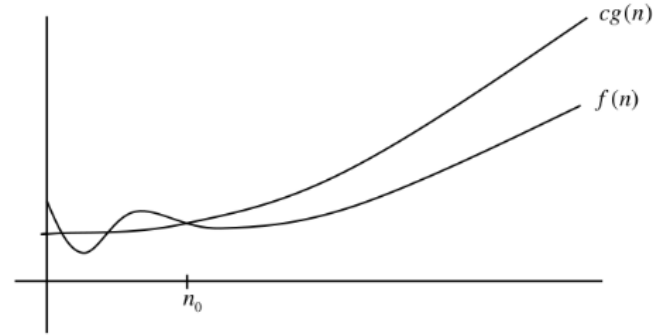
Time complexity

- **Exact runtime**
 - Exact number of operation the algorithm computes: $T(n) = 32*n + 16$
- **Asymptotic Complexity**
 - Different kind of Asymptotic Complexities
 - Simplified Exact runtime
 - Focuses on “The biggest offender”
 - $T(n) = 32*n + 16 \rightarrow O(n) = n$
- Tells us how long an algorithm will take to compute based on it's input length

Time complexity

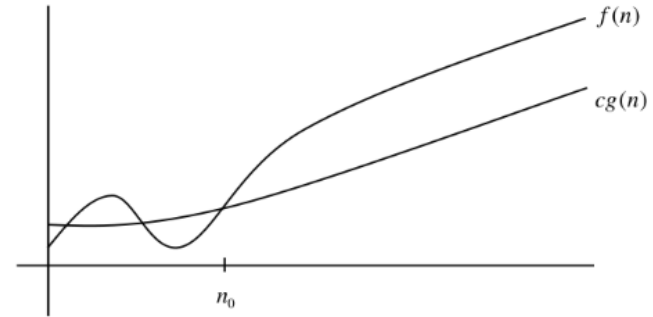
Big- O notation

- ▶ Asymptotic upper bound
- ▶ Used for worst-case analysis
- ▶ $f(n)$ and $g(n)$ are functions over non-negative integers
- ▶ $f(n) = O(g(n))$ if and only if there exist constants $c > 0$ and $n_0 > 0$ such that $f(n) \leq cg(n)$ for $n \geq n_0$



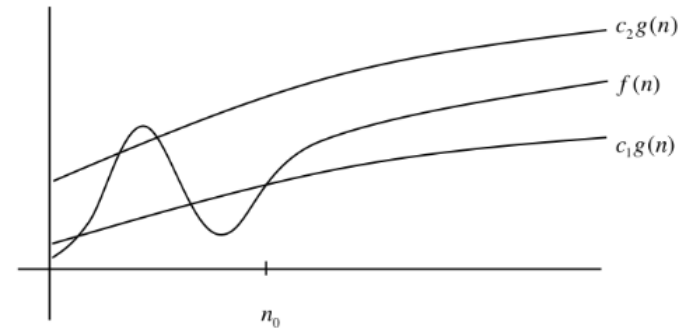
Big- Ω notation

- ▶ Asymptotic lower bound
- ▶ Used for best-case analysis
- ▶ $f(n) = \Omega(g(n))$ if and only if there exist constants $c > 0$ and $n_0 > 0$ such that $cg(n) \leq f(n)$ for $n \geq n_0$



Big Θ notation

- ▶ Asymptotic tight bound
- ▶ $f(n) = \Theta(g(n))$ if and only if there exist constants $c_1 > 0$, $c_2 > 0$, and $n_0 > 0$ such that $c_1g(n) \leq f(n) \leq c_2g(n)$ for $n \geq n_0$



We only care about the behavior of $f(n)$ and $g(n)$ after a certain n_0

Time complexity (cheat sheet)

different time complexity classes

low high										
80	$\log n$	$\log^2 n$	$\sqrt{n}$	n	$n \log n$	$n \log^2 n$	$n^{3/2}$	n^2	2^n	$n!$
80	3	9	3	10	30	90	30	100	1e4	4e6
80	6	36	10	100	600	3600	1000	1e4	1e30	9e157
80	9	81	31	1000	9000	8e5	3e4	1e6	1e301	∞
80	13	169	100	1e4	1e5	2e6	1e6	1e8	∞	∞
80	16	256	316	1e5	2e6	3e7	3e7	1e10	∞	∞
80	19	361	1000	1e6	2e7	4e8	1e9	1e12	∞	∞

► Arithmetic series

$$\text{► } \sum_{i=1}^n i = 1 + 2 + \dots + n = \frac{n(n+1)}{2}$$

► Geometric series for a real number $|a| < 1$:

$$\sum_{i=0}^{\infty} a^i = \frac{1}{1-a}$$

► Finite geometric series, integer n , real number $0 < a \neq 1$:

$$\sum_{i=0}^n a^i = 1 + a + a^2 + \dots + a^n = \frac{1-a^{n+1}}{1-a}$$

► Simple manipulations

$$\text{► } \sum_{i=1}^n ca_i = c \sum_{i=1}^n a_i$$

$$\text{► } \sum_{i=1}^n (a_i + b_i) = \sum_{i=1}^n a_i + \sum_{i=1}^n b_i$$

► Exponential function (see Taylor expansion):

$$e^x = \sum_{k=0}^{\infty} \frac{x^k}{k!} = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \dots$$

Logarithms and powers

$$\text{► } \log_a(bc) = \log_a b + \log_a c$$

$$\text{► } \log_a b = \log_c b / \log_c a$$

$$\text{► } \log_a b^c = c \log_a b$$

$$\text{► } a^m a^n = a^{m+n}$$

$$\text{► } a^{mn} = (a^m)^n$$

$$\text{► } a^{\log_a b} = b$$

$$\text{► } \lg b = \lg b = \log_2 b$$

Task 1.

- Exercise to familiarize yourself with the binary search
- Look at the solutions, if you still have question ask directly to me or on OLAT

Task 2, 3, 4, 5.

- Use asymptotic complexity
- Use the cheat sheet

Thank you for your attention.

**If you have any question write in the Olat forum or
come up to me.**